

Mongo DB

1) Illustrate CRUD operation in mongo DB with example in Python

[3 mark]

Code:-

```
from pymongo import MongoClient
# Connect to MongoDB
client = MongoClient('mongodb://localhost:27017/')
# Accessing the database
db = client['mydatabase']
# Accessing the collection
collection = db['mycollection']

# Create operation
data = {"name": "John", "age": 30, "city": "New York"}
collection.insert_one(data)
print("Data inserted successfully.")

# Read operation
query = {"name": "John"}
result = collection.find_one(query)
print("Read operation result:", result)

# Update operation
query = {"name": "John"}
new_values = {"$set": {"age": 35}}
collection.update_one(query, new_values)
print("Data updated successfully.")

# Read updated data
updated_result = collection.find_one(query)
print("Updated data:", updated_result)

# Delete operation
delete_query = {"name": "John"}
collection.delete_one(delete_query)
print("Data deleted successfully.")
```

2) write a python program to perform following operations. on MongoDB Database.

[5 mark]

- i) Create collection "Emp" with fields: Emp-name ,Emp-mobile, Emp, sal, Age.
- ii) Insert. 5 documents.
- iii) Find the employees getting salary between 5000 to 10000
- iv) Update mobile number for the employee named as "Riddhi"
- v) Display all employees in the order of "Age"

Code :-

```
from pymongo import MongoClient
# Connect to MongoDB
client = MongoClient('mongodb://localhost:27017/')
# Accessing the database
db = client['mydatabase']
# Accessing the collection
```

```

collection = db['Emp']
# Create collection "Emp" with fields
emp_data = [
    {"Emp-name": "John", "Emp-mobile": "1234567890", "Emp-sal": 8000, "Age": 35},
    {"Emp-name": "Alice", "Emp-mobile": "9876543210", "Emp-sal": 6000, "Age": 28},
    {"Emp-name": "Bob", "Emp-mobile": "8765432109", "Emp-sal": 10000, "Age": 40},
    {"Emp-name": "Riddhi", "Emp-mobile": "9999999999", "Emp-sal": 7000, "Age": 32},
    {"Emp-name": "Eve", "Emp-mobile": "1111111111", "Emp-sal": 12000, "Age": 25}
]
# Insert documents
collection.insert_many(emp_data)
print("Documents inserted successfully.")
# Find employees getting salary between 5000 to 10000
query = {"Emp-sal": {"$gte": 5000, "$lte": 10000}}
result = collection.find(query)
print("Employees with salary between 5000 to 10000:")
for employee in result:
    print(employee)
# Update mobile number for the employee named as "Riddhi"
update_query = {"Emp-name": "Riddhi"}
new_mobile = {"$set": {"Emp-mobile": "9365474578"}}
collection.update_one(update_query, new_mobile)
print("Mobile number updated for employee named Riddhi.")
# Display all employees in the order of "Age"
age_sorted_result = collection.find().sort("Age", 1)
print("Employees sorted by age:")
for employee in age_sorted_result:
    print(employee)

```

3) Write a python to create employee collection with field (ID ,name, address ,phone number, email and department)in mongo DB perform following operation [4 mark]

- i)display all employee in account department
- ii) delete employee with ID 210345
- iii) update phone with a new phone number for employee ID 123

Code:-

```

from pymongo import MongoClient
# Connect to MongoDB
client = MongoClient('mongodb://localhost:27017/')
# Accessing the database
db = client['mydatabase']

# Accessing the collection
collection = db['employee']
# Create employee collection with fields
employee_data = [
    {"ID": "123", "name": "John", "address": "123 Main St", "phone": "1234567890",
     "email": "john@example.com", "department": "account"},
    {"ID": "210345", "name": "Alice", "address": "456 Elm St", "phone": "9876543210",
     "email": "alice@example.com", "department": "finance"},

```

```

        {"ID": "789", "name": "Bob", "address": "789 Oak St", "phone": "5555555555",
         "email": "bob@example.com", "department": "account"},

        {"ID": "456", "name": "Eve", "address": "321 Maple St", "phone": "9999999999",
         "email": "eve@example.com", "department": "hr"}
    ]

# Insert documents
collection.insert_many(employee_data)
print("Documents inserted successfully.")

# Display all employees in account department
query_account = {"department": "account"}
account_department_employees = collection.find(query_account)

print("Employees in the account department:")
for employee in account_department_employees:
    print(employee)

# Delete employee with ID 210345
delete_query = {"ID": "210345"}
collection.delete_one(delete_query)
print("Employee with ID 210345 deleted successfully.")

# Update phone with a new phone number for employee ID 123
update_query = {"ID": "123"}
new_phone = {"$set": {"phone": "9876543210"}}
collection.update_one(update_query, new_phone)
print("Phone number updated for employee with ID 123.")

```

4) Write a program to retrieve and display employee details from employee collection stored in a Mongo DB database [3 mark]

Code:-

OR

```

from pymongo import MongoClient

client = MongoClient('mongodb://localhost:27017/')
db = client['mydatabase']
collection = db['employee']

# Retrieve and display employee details
employees = collection.find()

print("Employee details:")
for employee in employees:
    print("ID:", employee["ID"])
    print("Name:", employee["name"])
    print("Address:", employee["address"])
    print("Phone:", employee["phone"])
    print("Email:", employee["email"])
    print("Department:",
          employee["department"])
    print("-----")

from pymongo import MongoClient

client = MongoClient('mongodb://localhost:27017/')
db = client['mydatabase']
collection = db['employee']

# Retrieve and display employee details
employees = collection.find()

print("Employee details:")
for employee in employees:
    print("ID:", employee["ID"])
    print("Name:", employee["name"])
    print("Address:", employee["address"])
    print("Phone:", employee["phone"])
    print("Email:", employee["email"])
    print("Department:",
          employee["department"])
    print("-----")

```

5) Write a program to update employee details store in employee collection store in a Mongo DB database [3 mark]

Code:-

```
from pymongo import MongoClient

client = MongoClient('mongodb://localhost:27017/')
db = client['mydatabase']
collection = db['employee']

def update_employee(employee_id, update_field, new_value):
    # Construct update query
    update_query = {"ID": employee_id}
    new_data = {"$set": {update_field: new_value}}
    # Perform update operation
    update_result = collection.update_one(update_query, new_data)
    # Check if update was successful
    if update_result.modified_count > 0:
        print("Employee details updated successfully.")
        return True
    else:
        print("No employee found with the provided ID.")
        return False

# Prompt user for input
employee_id = input("Enter the ID of the employee you want to update: ")
update_field = input("Enter the field you want to update (name/address/phone/email/department): ")
new_value = input("Enter the new value: ")

# Call update function
update_employee(employee_id, update_field, new_value)
```

6) Write a Python program using Mongo DB database to create Books Collection having filleds - title, author (list), publisher, publisher address(a dict with keys like area, City, country), price, ISBN accept input from user to insert document [3 mark]

Code:-

```
from pymongo import MongoClient

client = MongoClient('mongodb://localhost:27017/')
db = client['mydatabase']
collection = db['Books']

def insert_book():
    # Accept input from user
    title = input("Enter the title of the book: ")
    author = input("Enter the author(s) of the book (separated by commas if multiple): ").split(',')
    publisher = input("Enter the publisher of the book: ")
    area = input("Enter the area of the publisher's address: ")
    city = input("Enter the city of the publisher's address: ")
    country = input("Enter the country of the publisher's address: ")
```

```

        price = float(input("Enter the price of the book: "))
        isbn = input("Enter the ISBN of the book: ")

# Construct document
    book_document = {
        "title": title,
        "author": author,
        "publisher": publisher,
        "publisher_address": {"area": area, "city": city, "country": country },
        "price": price,
        "ISBN": isbn
    }

# Insert document
    collection.insert_one(book_document)
    print("Book document inserted successfully.")

# Call insert_book function to insert document
    insert_book()

```

Regular Expression

- 1) Write a program to validate email address using regular expression also explain the meaning of each and every special character of a regular expression used by you in the program

[4 mark]

Code:-

```

import re

def validate_email(email):
# Regular expression pattern to validate email address
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
# Check if the email matches the pattern
    if re.match(pattern, email):
        return True
    else:
        return False
# Test the function
    email1 = "example@example.com"
    email2 = "invalid_email.com"

    print(validate_email(email1)) # Output: True
    print(validate_email(email2)) # Output: False

```

Explanation of the regular expression pattern:

- $\wedge$: This symbol marks the start of the string.

- **[a-zA-Z0-9._%+~!]{1,}**: This part matches the local part of the email address. It allows one or more occurrences of letters (both uppercase and lowercase), digits, and certain special characters like dot (.), underscore (_), percentage (%), plus (+), and hyphen (-).
- **@**: Matches the "@" symbol, which separates the local part from the domain part of the email address.
- **[a-zA-Z0-9.-]{1,}**: This part matches the domain name. It allows one or more occurrences of letters (both uppercase and lowercase), digits, dot (.), and hyphen (-).
- **\.**: Escapes the dot (.) character to match it literally, as it's a special character in regular expressions.
- **[a-zA-Z]{2,}**: This part matches the top-level domain (TLD), such as .com, .org, .net, etc. It allows two or more occurrences of letters (both uppercase and lowercase).
- **\$**: This symbol marks the end of the string.

Overall, the regular expression ensures that the email address follows a standard format, consisting of a local part, "@" symbol, domain name, and a valid top-level domain.

2) Write python program to check the validity of a password given by user The password should satisfy following criteria

- contain at least 1 letter between a & z
- contain at least 1 number between 0-9
- contain at least 1 letter between A & Z
- contain at least 1 character from \$, #, @, *
- Minimum length of password : 8
- Maximum length of password : 20

Code:-

```
import re

def validate_password(password):
    # Regular expression pattern to validate password
    pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[$#@*])[A-Za-z\d$#@*]{8,20}$'
    # Check if the password matches the pattern
    if re.match(pattern, password):
        return True
    else:
        return False

# Test the function
password1 = "Password@123"
password2 = "Weakpass1"

print(validate_password(password1)) # Output: True
print(validate_password(password2)) # Output: False
```

Explanation of the regular expression pattern:

- `^`: Asserts the start of the string.
- `(?=.*[a-z])`: Positive lookahead assertion to ensure the presence of at least one lowercase letter.
- `(?=.*[A-Z])`: Positive lookahead assertion to ensure the presence of at least one uppercase letter.
- `(?=.*\d)`: Positive lookahead assertion to ensure the presence of at least one digit.
- `(?=.*[$#@*])`: Positive lookahead assertion to ensure the presence of at least one special character from the set [`$`, `#`, `@`, `*`].
- `[A-Za-z\d$#@*]{8,20}`: Matches any combination of letters (both uppercase and lowercase), digits, and the specified special characters, with a length between 8 and 20 characters.
- `$`: Asserts the end of the string.

Overall, the regular expression ensures that the password satisfies all the given criteria: containing at least one lowercase letter, one uppercase letter, one digit, one special character, and having a length between 8 and 20 characters.

3) Write a program for extracting email address from given web page

[4 mark]

Code:-

```
import re
import requests

def extract_emails(url):
    # Fetch the web page content
    response = requests.get(url)
    if response.status_code == 200:
        # Extract email addresses using regex
        emails = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', response.text)
        return emails
    else:
        print("Failed to fetch the webpage")
        return []

# Example usage
url = "https://example.com"
emails = extract_emails(url)
print("Email addresses found on", url, ":", emails)
```

Replace `"https://example.com"` with the URL of the web page you want to extract email addresses from. This program uses the `requests` library to fetch the web page content and `re.findall()` function to extract email addresses using a regular expression pattern.

4) Write the program to validate URL using regular expression explain every special character of a regular expression used in this program [4 mark]

Code:-

```
import re

# Regular expression pattern to validate URL

def validate_url(url):
    pattern = re.compile(r'^(?:http|https):/(?:www\.)?[a-zA-Z0-9-]+\.[a-zA-Z]{2,}(?:\.[a-zA-Z]{2,})?/?[\s]*$')

    # Check if the URL matches the pattern
    if pattern.match(url):
        return True
    else:
        return False

# Example usage:
url = "https://www.example.com"
if validate_url(url):
    print("Valid URL")
else:
    print("Invalid URL")
```

Explanation of the regular expression used:

1. `^`: Matches the start of the string.
2. `(?:http|https):/`: Matches either "http://" or "https://".
3. `(?:www\.)?`: Matches an optional "www." at the beginning of the URL.
4. `[a-zA-Z0-9-]+`: Matches one or more alphanumeric characters or hyphens for the domain name.
5. `\.`: Matches a literal dot.
6. `[a-zA-Z]{2,}`: Matches two or more alphabetic characters for the top-level domain (TLD).
7. `(?:\.[a-zA-Z]{2,})?`: Matches an optional period followed by two or more alphabetic characters for additional subdomains.
8. `/?`: Matches an optional trailing slash.
9. `[\s]*$`: Matches any characters that are not whitespace until the end of the string.

This regular expression ensures that the URL follows the format of a typical web address, including the protocol, domain name, and optional subdomains, paths, and query parameters.

5) Write a Python program to remove all leading zero from an IP address and find the all five character long words in the string accept string from user

Code:-

```
import re

def remove_leading_zeros(ip_address):
    # Split the IP address by '.'
    parts = ip_address.split('.')
    # Remove leading zeros from each part and join them back
    cleaned_parts = [str(int(part)) for part in parts]
    cleaned_ip = '.'.join(cleaned_parts)
    return cleaned_ip

def find_five_char_words(input_string):
    # Find all words that are exactly five characters long
    words = re.findall(r'\b\w{5}\b', input_string)
    return words

# Accepting an IP address from the user
ip_address = input("Enter an IP address: ")
# Removing leading zeros from the IP address
cleaned_ip = remove_leading_zeros(ip_address)
print("IP address without leading zeros:", cleaned_ip)

# Accepting a string from the user
input_string = input("Enter a string: ")
# Finding all five-character long words in the string
five_char_words = find_five_char_words(input_string)
print("Five-character long words in the string:", five_char_words)
```

NumPy, Pandas, Matplotlib

NumPy

➤ Create 5x5 2D numpy array and retrieve top left corner 2x2 array from it.

Code:-

[2 mark]

```
import numpy as np
# Create a 5x5 2D NumPy array
array_5x5 = np.array([[1, 2, 3, 4, 5],
                       [6, 7, 8, 9, 10],
                       [11, 12, 13, 14, 15],
                       [16, 17, 18, 19, 20],
                       [21, 22, 23, 24, 25]])

# Retrieve the top left corner 2x2 array
```

```

top_left_corner = array_5x5[:2, :2]

print("Original 5x5 array:")
print(array_5x5)
print("\nTop left corner 2x2 array:")
print(top_left_corner)

```

OUTPUT:-

```

Original 5x5 array:
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]]

```

```

Top left corner 2x2 array:
[[1 2]
 [6 7]]

```

➤ **Write a program to illustrate slicing in numpy array.**

Code:-

[2 mark]

```

import numpy as np

# Create a 3x3 NumPy array
arr = np.array ( [[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]] )

# Print the original array
print("Original array:")
print(arr)

# Slicing to get a sub-array (2x2) from the original array
sub_arr = arr[:2, :2]

# Print the sub-array
print("\nSub-array (2x2) from the original array:")
print(sub_arr)

```

OUTPUT:-

```

Original array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Sub-array (2x2) from the original array:
[[1 2]
 [4 5]]

```

➤ **Create 3 x 3 numpy array and display column wise mean and median.**

Code:-

[2 mark]

```
import numpy as np
```

```
# Create a 3x3 NumPy array
```

```
arr = np.array([[1, 2, 3],  
               [4, 5, 6],  
               [7, 8, 9]])
```

```
# Calculate column-wise mean
```

```
column_means = np.mean(arr, axis=0)
```

```
# Calculate column-wise median
```

```
column_medians = np.median(arr, axis=0)
```

```
# Display results
```

```
print("Original array:")  
print(arr)  
print("\nColumn-wise mean:")  
print(column_means)  
print("\nColumn-wise median:")  
print(column_medians)
```

OUTPUT:-

Original array:

```
[[1 2 3]  
 [4 5 6]  
 [7 8 9]]
```

Column-wise mean:

```
[4. 5. 6.]
```

Column-wise median:

```
[4. 5. 6.]
```

➤ **Create a series from numpy array and find the max and mean of a unique elements of series.**

Code:-

[2 mark]

```
import pandas as pd
```

```
import numpy as np
```

```
# Create a NumPy array
```

```
arr = np.array( [1, 2, 3, 4, 5, 1, 2, 3, 4, 5] )
```

```
# Create a pandas Series from the NumPy array
```

```
series = pd.Series(arr)
```

```
# Find unique elements in the Series and calculate their maximum and mean
```

```
unique_elements = series.unique()
```

```
max_value = unique_elements.max()
```

```
mean_value = unique_elements.mean()
```

```
# Display results
```

```
print("Series:")
```

```

print(series)
print("\nUnique elements:", unique_elements)
print("\nMaximum of unique elements:", max_value)
print("Mean of unique elements:", mean_value)

```

OUTPUT:-

```

Series:
0  1
1  2
2  3
3  4
4  5
5  1
6  2
7  3
8  4
9  5
dtype: int64

Unique elements: [1 2 3 4 5]
Maximum of unique elements: 5
Mean of unique elements: 3.0

```

➤ Write the program to illustrate number attribute

- | | |
|--------------------|------------------------|
| i) ndarray.shape() | ii) np.zeros() |
| iii) np.eye() | iv) np.random.random() |

Code:-

[4 mark]

```

import numpy as np

# i) ndarray.shape()
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Shape of the array:", arr.shape)

# ii) np.zeros()
zeros_arr = np.zeros((2, 3))      # Creates a 2x3 array filled with zeros
print("Array filled with zeros:")
print(zeros_arr)

# iii) np.eye()
eye_arr = np.eye(3)               # Creates a 3x3 identity matrix
print("Identity matrix:")
print(eye_arr)

# iv) np.random.random()
random_arr = np.random.random((2, 2))      # Creates a 2x2 array with random values between 0 and 1
print("Array filled with random values:")
print(random_arr)

```

Pandas

❖ Write a program to reverse each words of *data.txt* file using pandas

Code:-

[3 mark]

```
import pandas as pd
def reverse_words(file_name):
    try:
# Read the file into a DataFrame
        df = pd.read_csv(file_name, header=None)
# Reverse each word in the DataFrame
        reversed_df = df.applymap(lambda word: word[::-1])
# Write the reversed words back to the file
        reversed_df.to_csv(file_name, index=False, header=False, sep=' ', na_rep="", mode='w')
        print("Words reversed successfully.")

    except FileNotFoundError:
        print(f"File '{file_name}' not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

# Example usage:
file_name = "data.txt"          # Replace with the name of your file
reverse_words(file_name)
```

❖ Create pandas dataprame using 2D list perform following Operation

- i) count number of rows
- ii) Count "missing values in first Column
- iii) Display number of columns in data frame

Code:-

[4 mark]

```
import pandas as pd
# Create a 2D list
data = [[1, 2, 3],
        [4, None, 6],
        [7, 8, 9]]
# Create DataFrame
df = pd.DataFrame(data, columns=['A', 'B', 'C'])

# i) Count number of rows
num_rows = len(df)
# ii) Count missing values in the first column
missing_values_first_column = df['A'].isnull().sum()
# iii) Display number of columns in the DataFrame
num_columns = len(df.columns)

# Print results
print("DataFrame:")
print(df)
print("\nNumber of rows:", num_rows)
print("Missing values in the first column:", missing_values_first_column)
```

```
print("Number of columns:", num_columns)
```

OUTPUT:-

DataFrame:

	A	B	C
0	1	2.0	3
1	4	NaN	6
2	7	8.0	9

Number of rows: 3

Missing values in the first column: 0

Number of columns: 3

❖ Prepare the Pandas dataframe from CSV file perform following operation

i) fill all NaN values with a mean of respective to column.

ii) Display the last five rows

Code:-

[2 mark]

```
import pandas as pd
```

```
# Load the CSV file into a DataFrame
```

```
df = pd.read_csv('your_file.csv')
```

```
# Fill NaN values with mean of respective column
```

```
df.fillna(df.mean(), inplace=True)
```

```
# Display the last five rows
```

```
print(df.tail())
```

❖ # Create the DataFrame

```
import pandas as pd
```

```
data = {  
    'ID': [1001, 2002, 1003],  
    'Name': ['Mohan', 'Sachin', 'Virat'],  
    'HRA': [12000, 13000, 11000],  
    'TA': [6000, 5000, 4000],  
    'DA': [10000, 9000, 8000]  
}
```

```
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
# Compute sum of each column
```

```
column_sums = df.sum()
```

```
print("\nSum of Each Column:")
```

```
print(column_sums)
```

Compute mean of each integer column

```
integer_columns_mean = df[['ID', 'HRA', 'TA', 'DA']].mean()
print("\nMean of Each Integer Column:")
print (integer_columns_mean)
```

Compute median of each integer column

```
integer_columns_median = df[['ID', 'HRA', 'TA', 'DA']].median()
print("\nMedian of Each Integer Column:")
print (integer_columns_median)
```

Matplotlib

❖ Draw line graph using Matplotlib and decorate it by adding various elements used suitable data

Code:-

[4 mark]

```
import matplotlib.pyplot as plt
```

Sample data

```
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales_2023 = [10000, 12000, 11000, 13000, 14000, 15000]
sales_2024 = [11000, 13000, 12000, 14000, 15000, 16000]
```

Plotting the data

```
plt.plot(months, sales_2023, marker='o', label='2023 Sales')
plt.plot(months, sales_2024, marker='o', label='2024 Sales')
```

Adding labels and title

```
plt.xlabel('Months')
plt.ylabel('Sales')
plt.title('Sales Comparison between 2023 and 2024')
```

Adding grid

```
plt.grid(True)
```

Adding legend

```
plt.legend()
```

Adding annotations

```
for i in range(len(months)):
    plt.text(months[i], sales_2023[i], str(sales_2023[i]), ha='center', va='bottom')
    plt.text(months[i], sales_2024[i], str(sales_2024[i]), ha='center', va='bottom')
```

Display the plot

```
plt.show()
```

❖ **Draw bar graph using Matplotlib and decorate it by adding various element**

Code:-

[3 mark]

```
import matplotlib.pyplot as plt

# Sample data
categories = ['Category A', 'Category B', 'Category C', 'Category D']
values = [25, 40, 30, 35]

# Plotting the bar graph
plt.bar(categories, values, color='skyblue')

# Adding labels and title
plt.xlabel('Categories')
plt.ylabel('Values')
plt.title('Bar Graph Example')

# Adding grid
plt.grid(True)

# Adding annotations
for i in range(len(categories)):
    plt.text(categories[i], values[i], str(values[i]), ha='center', va='bottom')

# Display the plot
plt.show()
```

Programs

❖ **Write a program to read the content of file and display the occurrence of given character**

Code:-

[3 mark]

```
def count_occurrence(file_name, char):
    try:
        with open(file_name, 'r') as file:
            content = file.read()
            occurrence = content.count(char)
            print(f"The character '{char}' occurs {occurrence} times in the file.")
    except FileNotFoundError:
        print(f"File '{file_name}' not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

# Example usage:
file_name = "example.txt"
character = "a"
count_occurrence(file_name, character)
```

Replace with the name of your file
Replace with the character you want to count

Replace **"example.txt"** with the name of your file and **"a"** with the character you want to count.

❖ **Write a program to accept decimal number and print its octal and hexadecimal equivalent in python**

Code:-

[3 mark]

```
# Convert decimal to octal and hexadecimal
def decimal_to_octal_hex(decimal):
    try:
        octal = oct(decimal)
        hexadecimal = hex(decimal)
        print(f"The octal equivalent of {decimal} is: {octal}")
        print(f"The hexadecimal equivalent of {decimal} is: {hexadecimal}")
    except Exception as e:
        print(f"An error occurred: {e}")

# Accept decimal number from the user
decimal_number = int(input("Enter a decimal number: "))
# Convert and print equivalents
decimal_to_octal_hex(decimal_number)
```

❖ **Write a program to check if the input year is leap year or not validate the input**

Code:-

[3 mark]

```
def is_leap_year(year):
    try:
# Validate input
        if year < 0:
            print("Please enter a positive year.")
            return False
# Check if it's a leap year
        if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
            print(f"{year} is a leap year.")
        else:
            print(f"{year} is not a leap year.")
    except ValueError:
        print("Please enter a valid integer year.")

# Accept year from the user
    try:
        input_year = int(input("Enter a year: "))
        is_leap_year(input_year)
    except ValueError:
        print("Please enter a valid integer year.")
```

❖ **Write a program to print the sum of natural number using recursion**

Code:-

[3 mark]

```
def sum_of_natural_numbers(n):
    if n <= 0:
        return 0
    else:
        return n + sum_of_natural_numbers(n - 1)
```

```
# Accept input from the user
try:
    n = int(input("Enter a positive integer: "))
    if n < 0:
        print("Please enter a positive integer.")
    else:
        result = sum_of_natural_numbers(n)
        print(f"The sum of first {n} natural numbers is: {result}")
except ValueError:
    print("Please enter a valid integer.")
```

❖ **write a program to to reverse ach word of "data.txt" file**

Code:-

[2 mark]

```
def reverse_words(file_name):
    try:
        with open(file_name, 'r') as file:
            content = file.read()
            words = content.split() # Split the content into words

    # Reverse each word
        reversed_words = [word[::-1] for word in words]

    # Join the reversed words back into a string
        reversed_content = ' '.join(reversed_words)

    # Write the reversed content to a new file
        with open("reversed_data.txt", 'w') as reversed_file:
            reversed_file.write(reversed_content)

        print("Reversed content has been written to 'reversed_data.txt'.")

    except FileNotFoundError:
        print("File not found.")

    except Exception as e:
        print("An error occurred:", e)

# Example usage:
file_name = "data.txt" # Replace this with the name of your file
reverse_words(file_name)
```

❖ **Write a multithreading program where one thread prints square of a number another thread prints factorial of a number also display the total time taken for the execution in python**

Code:-

[4 mark]

```
import threading
import time
import math
```

```

def calculate_square(num):
    print(f"Square of {num}: {num**2}")

def calculate_factorial(num):
    print(f"Factorial of {num}: {math.factorial(num)}")

def main():
    num = int(input("Enter a number: "))
    start_time = time.time()
    square_thread = threading.Thread(target=calculate_square, args=(num,))
    factorial_thread = threading.Thread(target=calculate_factorial, args=(num,))
    square_thread.start()
    factorial_thread.start()
    square_thread.join()
    factorial_thread.join()
    end_time = time.time()
    total_time = end_time - start_time
    print(f"Total time taken for execution: {total_time} seconds")

if __name__ == "__main__":
    main()

```

❖ What is a anonym of function in Python demonstrate with example

Code:-

[2 mark]

In Python, an anonymous function is a function defined using the **lambda** keyword, which allows you to create a small, one-line function without a formal name. Here's an example:

```

# Example of an anonymous function (lambda function)
addition = lambda x, y: x + y

# Using the anonymous function
result = addition(5, 3)
print("Result of addition:", result)

```

In this example, **lambda x, y: x + y** creates an anonymous function that takes two arguments **x** and **y** and returns their sum. The function is assigned to the variable **addition**, and then it's called with arguments **5** and **3**, resulting in **8**.

❖ Write a program to check the number entered, by user is even or odd program should accept integer digits only

Code:-

[3 mark]

```

def check_even_odd():
    while True:
        try:
            # Prompt the user to enter an integer
            num = int(input("Enter an integer: "))

            # Check if the number is even or odd

```

```

        if num % 2 == 0:
            print("The number is even.")
        else:
            print("The number is odd.")
    # Exit the loop if an integer is entered successfully
    break
except ValueError:
    # If the entered value is not an integer, display an error message
    print("Please enter an integer only.")

# Call the function to check even or odd
check_even_odd()

```

❖ write a python function to check the given number is even or odd handle Suitable exceptions.

Code:-

[3 mark]

```

def check_even_odd(num):
    try:
        # Convert the input to an integer
        num = int(num)

        # Check if the number is even or odd
        if num % 2 == 0:
            return "Even"
        else:
            return "Odd"
    except ValueError:
        # Handle the exception if the input cannot be converted to an integer
        return "Invalid input: Please enter an integer."

# Example usage:
num_input = input("Enter a number: ")
result = check_even_odd(num_input)
print(result)

```

❖ Write a Python program to find the factorial of given number using recursion

Code:-

[2 mark]

```

def factorial(n):
    # Base case: if n is 0 or 1, factorial is 1
    if n == 0 or n == 1:
        return 1
    # Recursive case: factorial of n is n * factorial of (n-1)
    else:
        return n * factorial(n-1)

# Accepting a number from the user
number = int(input("Enter a number: "))

# Calculating factorial using recursion
result = factorial(number)
print("Factorial of", number, "is", result)

```

❖ Write a program to check whether the interesting and number is palindrome or not

Code:-

[2 mark]

```
def is_palindrome(num):
    # Convert number to string for easier manipulation
    num_str = str(num)
    # Compare the string with its reverse
    return num_str == num_str[::-1]

# Accepting a number from the user
number = int(input("Enter a number: "))

# Checking if the number is a palindrome
if is_palindrome(number):
    print(number, "is a palindrome.")
else:
    print(number, "is not a palindrome.")
```

❖ Write a program to accept string from user and display the same sting after removing vowel from it in Python

Code:-

[2 mark]

```
def remove_vowels(string):
    vowels = 'aeiouAEIOU'
    return "".join(char for char in string if char not in vowels)

user_input = input("Enter a string: ")
result = remove_vowels(user_input)
print("String after removing vowels:", result)
```

Thread & I/O File Operation

❖ Write python program to read employee.txt file and display alternate employee records

Code:-

[2 mark]

```
# Open the file in read mode
with open('employee.txt', 'r') as file:
    # Read all lines from the file
    lines = file.readlines()
    # Display alternate employee records
    for i in range(0, len(lines), 2):
        print(lines[i].strip())
```

- ❖ Write a program for synchronization of Thread using RLOCK accept the two numbers from user and calculate factorial of both number simultaneously

Code:-

```
import threading

# Function to calculate factorial
def factorial(n, lock):
    lock.acquire() # Acquire the lock
    result = 1

    for i in range(1, n+1):
        result *= i
    print("Factorial of", n, "is", result)
    lock.release() # Release the lock

# Accepting two numbers from the user
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))

# Creating an RLock
lock = threading.RLock()

# Creating and starting threads to calculate factorial of each number
thread1 = threading.Thread(target=factorial, args=(num1, lock))
thread2 = threading.Thread(target=factorial, args=(num2, lock))

thread1.start()
thread2.start()

# Wait for both threads to finish before continuing main program
thread1.join()
thread2.join()
```

- ❖ Write a Python program to demonstrate

- i) creating thread without using any class
- ii) creating thread by extending thread class

Code:-

[4 mark]

Creating a Thread without Using any Class:

```
import threading

# Function to be executed by the thread
def print_numbers():
    for i in range(1, 6):
        print("Thread without class:", i)

# Creating and starting the thread
thread1 = threading.Thread(target=print_numbers)
thread1.start()
thread1.join() # Wait for the thread to finish before continuing main program
```

Creating a Thread by Extending the Thread Class:

```
import threading

# Define a new thread class by extending the Thread class
class MyThread(threading.Thread):
    def __init__(self):
        threading.Thread.__init__(self)

    # Override the run() method to define the behavior of the thread
    def run(self):
        for i in range(1, 6):
            print("Thread with class extension:", i)

# Creating and starting the thread
thread2 = MyThread()
thread2.start()
thread2.join()          # Wait for the thread to finish before continuing main program
```

❖ Develop of Python program to remove the comment character from all the line in the given file accept the file name from user

Code:-

[3 mark]

```
def remove_comments(filename):
    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
            # Remove comment characters from each line
            lines_without_comments = [line.split('#')[0].rstrip() for line in lines]
            # Print the lines without comments
            for line in lines_without_comments:
                print(line)
    except FileNotFoundError:
        print("File not found.")

# Accepting file name from the user
filename = input("Enter the file name: ")
# Removing comments from the file
remove_comments(filename)
```

❖ Developer Python program to print number of line words and character present in the given file accept the file name from user Ander necessary exception

Code:-

[3 mark]

```
def count_lines_words_chars(filename):
    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
            num_lines = len(lines)
```

```
num_words = sum(len(line.split())) for line in lines)
num_chars = sum(len(line) for line in lines)
print("Number of lines:", num_lines)
print("Number of words:", num_words)
print("Number of characters:", num_chars)
except FileNotFoundError:
    print("File not found.")

# Accepting file name from the user
filename = input("Enter the file name: ")
# Counting lines, words, and characters
count_lines_words_chars(filename)
```